

FOCUSFINGERPRINT

Personal Study Pattern Analyzer

PYTHON ESSENTIALS PROJECT REPORT

Submitted By

Khwahish Agrawal

Registration Number

24MIP10167

University

VIT Bhopal University

Project Type

Command-Line Python Application

Submission Date

30 September 2026

GitHub Repository

github.com/Khwahish-Agrawal/FocusFingerprint

Table of Contents

FOCUSFINGERPRINT	4
Personal Study Pattern Analyzer	4
Python Essentials Project Report	4
1. Introduction	4
Objectives	4
2. Problem Statement	5
Project Scope.....	5
Target Users.....	5
High-Level Features	5
3. Functional Requirements	6
4. Non-Functional Requirements	6
5. System Architecture	6
Architecture Flow	7
Main Components.....	7
main.py	7
session_manager.py	7
data_handler.py.....	7
analyzer.py	7
fingerprint.py	7
report_generator.py	7
6. Design Diagrams	7
6.1 Workflow	8
6.2 Storage Design	9
7. Design Decisions and Rationale	10
7.1 Command-Line Interface	10
7.2 CSV Storage	10
7.3 Modular Structure.....	10
7.4 FocusFingerprint Logic.....	10
8. Implementation Details	10
8.1 Project Modules.....	10
8.2 Processing Flow.....	11
9. Screenshots / Results.....	11
Screenshot 1 – Main Menu	11

Screenshot 2 – Focus Analysis	11
Screenshot 3 – Generated Report.....	12
10. FocusFingerprint Result.....	12
11. Generated Report.....	13
12. Testing Approach	13
Test Command	13
Test Result	13
Result	14
13. Challenges Faced.....	14
14. Learnings and Key Takeaways	14
15. Future Enhancements	14
16. GitHub Repository and Project Structure	15
GitHub Repository	15
Project Structure	15
17. References	15
Final Page.....	16
Thank You	16

FOCUSFINGERPRINT

Personal Study Pattern Analyzer

Python Essentials Project Report

Submitted By:

Khwahish Agrawal

Registration Number:

24MIP10167

University:

VIT Bhopal University

Project Type:

Command-Line Python Application

GitHub Repository:

<https://github.com/Khwahish-Agrawal/FocusFingerprint>

Submission Date:

30 September 2026

1. Introduction

FocusFingerprint is a command-line Python application developed to help students record their study sessions and understand their observed study patterns.

The application records information such as date, subject, study duration, number of distractions, focus rating, and study period. The recorded information is stored in a CSV file and processed using Python functions and modules.

The project applies core Python programming concepts including variables, data types, lists and dictionaries, conditions, loops, functions, modules, file handling, CSV processing, exception handling, data processing, calculations, and automated testing.

Objectives

- Provide a simple way to record study sessions.
- Store study-session data in a structured CSV file.
- Validate user input and handle invalid values.
- Calculate useful study statistics.
- Identify observable study patterns.
- Generate a personal FocusFingerprint.
- Generate a text-based study report.
- Demonstrate modular Python implementation and automated testing.

2. Problem Statement

Students often study different subjects for different amounts of time and at different times of the day. Their focus and number of distractions can also change from one study session to another.

However, students may not have a simple way to record these sessions and identify patterns in their study habits.

FocusFingerprint provides a command-line solution where students can record their study sessions and analyze their study duration, focus level, distractions, subject, and study period.

The application processes the collected data and generates a personal FocusFingerprint that summarizes the user's observed study patterns.

Project Scope

The scope of FocusFingerprint includes:

- Recording individual study sessions.
- Storing study-session data in a CSV file.
- Validating user inputs.
- Handling invalid input.
- Viewing previously recorded study sessions.
- Calculating total study time.
- Calculating average focus.
- Calculating average distractions.
- Identifying the subject with the highest focus.
- Identifying the study period with the highest focus.
- Identifying the most distracting subject.
- Identifying the session length associated with the highest focus.
- Measuring study consistency.
- Generating a personal FocusFingerprint.
- Generating a text-based study report.
- Testing analysis functions using Python's `unittest` framework.

The current version is a command-line application and does not include a graphical user interface or an external database.

Target Users

- School students.
- College and university students.
- Students preparing for examinations.
- Learners who want to track and understand their study habits.

High-Level Features

1. Study Session Recording
2. Study Data Storage
3. Study Session Viewing
4. Focus Analysis

5. FocusFingerprint Generation
6. Report Generation
7. Automated Testing

3. Functional Requirements

ID	Functional Requirement
FR1	The system shall allow the user to add a study session.
FR2	The system shall validate duration, distractions, and focus-rating inputs.
FR3	The system shall store study sessions in a CSV file.
FR4	The system shall allow users to view previously recorded sessions.
FR5	The system shall calculate total study duration, average focus, and average distractions.
FR6	The system shall identify study patterns based on subject and study period.
FR7	The system shall generate a personal FocusFingerprint.
FR8	The system shall generate and save a text-based study report.
FR9	The system shall handle situations where no study data is available.
FR10	The system shall provide automated tests for analysis functions.

4. Non-Functional Requirements

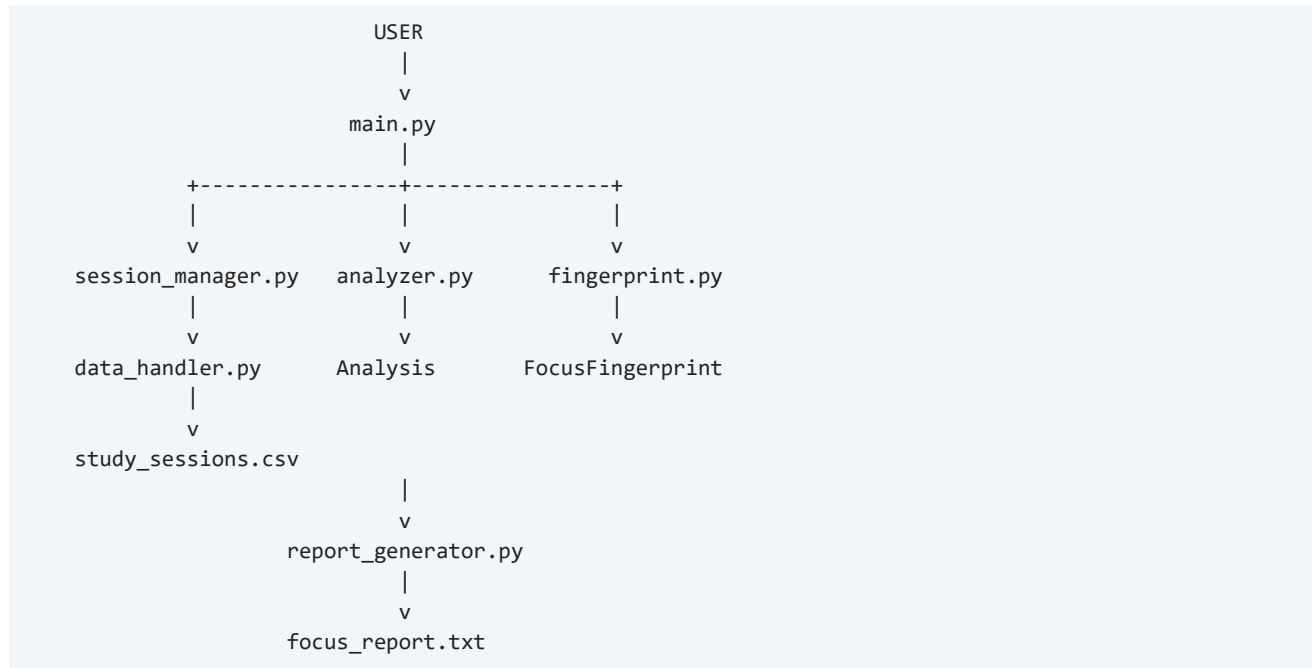
ID	Non-Functional Requirement
NFR1	**Usability:** The application should provide a simple numbered command-line menu.
NFR2	**Performance:** The application should process personal study data efficiently.
NFR3	**Reliability:** Invalid numeric input should be handled without terminating the application.
NFR4	**Maintainability:** Functionality should be divided into separate modules.
NFR5	**Data Integrity:** Study-session fields should be stored consistently in the CSV file.
NFR6	**Portability:** The application should run using standard Python libraries.
NFR7	**Testability:** Analysis functions should be tested using automated tests.
NFR8	**Simplicity:** The current version should not require external Python packages.

5. System Architecture

FocusFingerprint follows a modular command-line architecture.

The main program controls the application menu and communicates with different modules responsible for session management, data storage, analysis, fingerprint generation, and report generation.

Architecture Flow



Main Components

main.py

Controls the main menu and connects the different modules.

session_manager.py

Collects study-session information and validates user input.

data_handler.py

Handles saving and loading study-session information from the CSV file.

analyzer.py

Performs calculations and identifies study patterns.

fingerprint.py

Generates the user's FocusFingerprint.

report_generator.py

Creates and saves the final text report.

6. Design Diagrams

The project contains the following design artifacts in the docs folder:

- `architecture.png`
- `workflow.png`
- `use\_case.png`
- `class\_diagram.png`
- `sequence\_diagram.png`
- `storage\_diagram.png`

6.1 Workflow

```

Start
|
v
Display Main Menu
|
v
User Selects Option
|
+---- Add Session
|   |
|   v
|   Validate Input
|   |
|   v
|   Save to CSV
|
+---- View Sessions
|   |
|   v
|   Load CSV Data
|
+---- Analyze Focus
|   |
|   v
|   Calculate Statistics
|
+---- Generate Fingerprint
|   |
|   v
|   Analyze Study Patterns
|
+---- Generate Report
|   |
|   v
|   Save Report
|   |
|   v
|   End

```

6.2 Storage Design

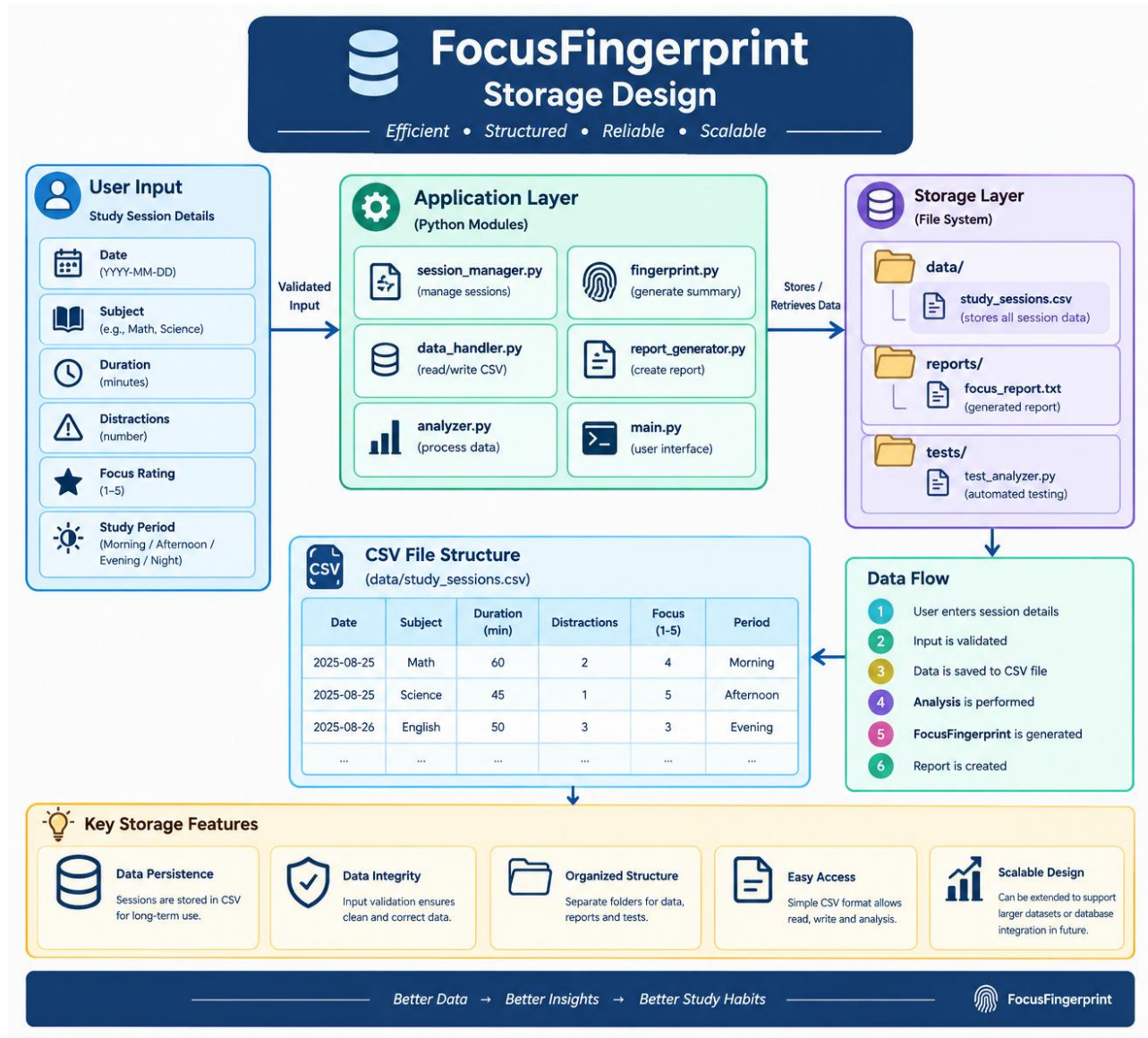


Figure 6.2: FocusFingerprint storage design — CSV-based study-session data flow.

The study sessions are stored in:

```
data/study_sessions.csv
```

The CSV contains:

Field	Description
date	Date of study session
subject	Subject studied
duration	Study duration in minutes
distractions	Number of distractions
focus_rating	Focus rating from 1 to 5
period	Morning, Afternoon, Evening, or Night

The generated report is stored in:

```
reports/focus_report.txt
```

7. Design Decisions and Rationale

7.1 Command-Line Interface

A command-line interface was selected because the project focuses on demonstrating Python programming concepts and can be executed directly from a terminal without requiring a graphical interface.

7.2 CSV Storage

CSV was selected as a simple structured storage format for personal study-session records.

Python's standard csv module is sufficient for reading and writing the data.

7.3 Modular Structure

The application is divided into different modules instead of placing all functionality in a single Python file.

This separates:

- Input handling
- Data storage
- Data analysis
- Fingerprint generation
- Report generation
- Application control

This makes the project easier to understand and maintain.

7.4 FocusFingerprint Logic

The FocusFingerprint summarizes observed values from recorded sessions.

It considers:

- Strongest subject
- Best study period
- Best session length
- Main distraction area
- Average focus

The FocusFingerprint is a project-level descriptive summary and is not intended to be a scientific diagnosis of learning ability.

8. Implementation Details

8.1 Project Modules

File	Purpose
`main.py`	Controls the menu and application

`session_manager.py`	Captures and validates study-session input
`data_handler.py`	Reads and writes CSV data
`analyzer.py`	Calculates statistics and study patterns
`fingerprint.py`	Generates the FocusFingerprint
`report_generator.py`	Creates and saves the text report
`tests/test_analyzer.py`	Contains automated tests
`statement.md`	Contains problem statement, scope, users, and features
`requirements.md`	Contains functional and non-functional requirements

8.2 Processing Flow

1. The user selects an option from the main menu.
2. Study-session information is collected.
3. Numeric values are validated.
4. Valid data is stored in the CSV file.
5. Stored sessions are loaded when analysis is requested.
6. The analyzer calculates statistics.
7. Study patterns are identified.
8. The FocusFingerprint is generated.
9. A text report can be generated and saved.

9. Screenshots / Results

Screenshot 1 – Main Menu

```
=====
          FOCUSFINGERPRINT
=====
Personal Study Pattern Analyzer

1. Add Study Session
2. View Study Sessions
3. Analyze Focus
4. Generate FocusFingerprint
5. Generate Report
6. Exit

Enter your choice: █
```

Caption:

Figure 1: FocusFingerprint main menu displayed in the command line.

Screenshot 2 – Focus Analysis

The application produced the following analysis:

```

Enter your choice: 3

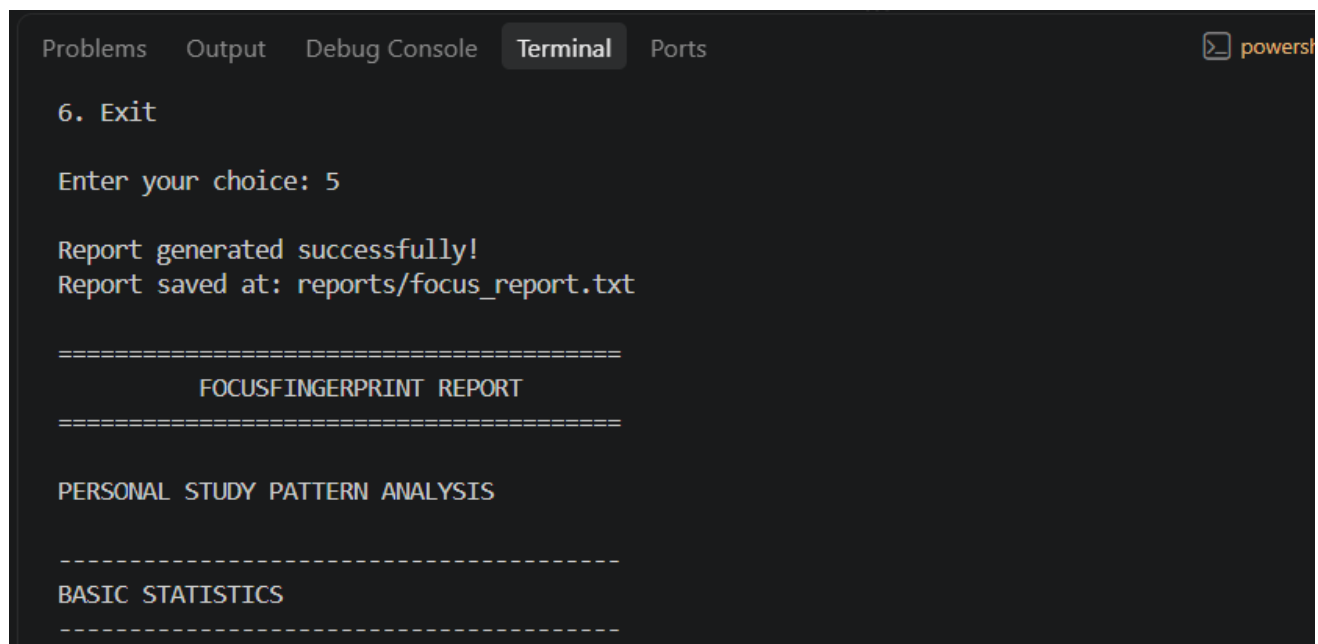
--- Focus Analysis ---
Total study time: 150 minutes
Average focus: 4.5/5
Average distractions: 2.5
Best subject for focus: DBMS
Best study period: Evening
Most distracting subject: Python
Best session length: 60 minutes
Study consistency: 100.0%

```

Caption:

Figure 2: Focus analysis generated from recorded study sessions.

Screenshot 3 – Generated Report



```

Problems  Output  Debug Console  Terminal  Ports
6. Exit

Enter your choice: 5

Report generated successfully!
Report saved at: reports/focus_report.txt

=====
                FOCUSFINGERPRINT REPORT
=====

PERSONAL STUDY PATTERN ANALYSIS

-----
BASIC STATISTICS
-----

```

Caption:

Figure 3: Generated FocusFingerprint report.

10. FocusFingerprint Result

The application generated the following FocusFingerprint:

```

=====
                YOUR FOCUSFINGERPRINT
=====

Strongest Subject      : DBMS
Best Study Period      : Evening
Best Session Length    : 60 minutes
Main Distraction Area  : Python
Average Focus          : 4.50/5

```

```

-----
Your study pattern shows that your
focus varies according to subject,
study period and session length.
=====

```

11. Generated Report

The application successfully generated:

```
reports/focus_report.txt
```

The report contains:

- Basic statistics
- Average focus
- Average distractions
- Best subject
- Best study period
- Best session length
- Most distracting subject
- Study consistency
- FocusFingerprint summary

12. Testing Approach

Automated testing was implemented using Python's built-in unittest framework.

The following areas were tested:

- Basic statistics
- Best subject
- Best study period
- Most distracting subject
- Best session length
- Study consistency
- Empty-data handling

Test Command

```
python -m unittest tests/test_analyzer.py
```

Test Result

```

.....
-----
Ran 7 tests in 0.001s

```

OK

Result

7 out of 7 tests passed successfully.

Manual execution was also performed to verify:

- Main menu
- Focus analysis
- FocusFingerprint generation
- Report generation

13. Challenges Faced

During the development of FocusFingerprint, the following challenges were encountered:

1. Designing a meaningful project within the Python Essentials syllabus.
2. Dividing the project into multiple modules instead of using one large file.
3. Handling invalid numeric input.
4. Converting CSV values from strings before performing calculations.
5. Handling empty datasets safely.
6. Organizing source code, documentation, tests, data, reports, and diagrams.
7. Setting up Git and publishing the project on GitHub.

14. Learnings and Key Takeaways

Through this project, I learned:

- How to divide a Python project into multiple modules.
- How to use functions effectively.
- How to work with lists, dictionaries, and sets.
- How to read and write CSV files.
- How to use exception handling for input validation.
- How to perform calculations on stored data.
- How to group and compare data for simple pattern analysis.
- How to write automated tests using `unittest`.
- How to organize a software project.
- How to use Git for version control.
- How to create and push a project to GitHub.
- How to document a project using README and project documentation.

15. Future Enhancements

The following features can be added in future versions:

- Weekly and monthly study analysis.
- Graphs for focus and distraction trends.
- More detailed consistency calculations.

- Editing existing study sessions.
- Deleting study sessions.
- Configurable study periods.
- Additional report formats.
- Graphical user interface.
- Database storage for larger datasets.

16. GitHub Repository and Project Structure

The project is maintained in a public GitHub repository using Git for version control.

GitHub Repository

<https://github.com/Khwahish-Agrawal/FocusFingerprint>

Project Structure

```
FocusFingerprint/
├── README.md
├── statement.md
├── requirements.md
├── main.py
├── session_manager.py
├── data_handler.py
├── analyzer.py
├── fingerprint.py
├── report_generator.py
├── data/
│   └── study_sessions.csv
├── reports/
│   └── focus_report.txt
├── tests/
│   └── test_analyzer.py
└── docs/
    ├── architecture.png
    ├── workflow.png
    ├── use_case.png
    ├── class_diagram.png
    ├── sequence_diagram.png
    └── storage_diagram.png
```

17. References

8. ****VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines.****
9. ****Python Standard Library Documentation**** – concepts related to `csv`, `os`, and `unittest`.
10. ****FocusFingerprint GitHub Repository**** – source code and project documentation.

Final Page

Thank You

FocusFingerprint – Personal Study Pattern Analyzer

Khwahish Agrawal

24MIP10167

VIT Bhopal University